pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Case Study: bondmath through CI

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: bondmath through CI

Today

1. ci_history: the runs as a table
2. pass_rate: what counts as a pass
3. Case study: the files that ship, and never
4. Three scans before the first push
5. A pull request that breaks DV01: red on Linux
6. The fix, green, merged, tagged v1.0
7. The tests against Excel, row by row
8. What green proves, and where it fails

Two functions

`ci_history`

gh run list as a table

`pass_rate`

the share that passed

No notebook cell reaches GitHub: the runs are shown from their masked transcripts.

How ci_history works

The problem	A desk that ships through CI needs the history of runs, not one run at a time
The method	gh run list --json prints one object for each run; skip to the first [and parse
The rows	Run number, branch, event, conclusion, start time in UTC, oldest first
You control	Which fields gh is asked for
What comes out	A table to group and chart; two runs of one commit are two rows

ci_history: the runs as a table

```
run           branch        event conclusion                 created
38236890266             main         push    failure 2026-10-12 10:31:41+00:00
38236944176             main         push    failure 2026-10-12 10:32:12+00:00
38237082499             main         push    success 2026-10-12 10:33:33+00:00
38239178703 dv01-clean-price         push    failure 2026-10-12 10:53:52+00:00
38239184167 dv01-clean-price pull_request    failure 2026-10-12 10:53:55+00:00
38239244347 dv01-clean-price         push    success 2026-10-12 10:54:30+00:00
38239249431 dv01-clean-price pull_request    success 2026-10-12 10:54:34+00:00
38239305566             main         push    success 2026-10-12 10:55:06+00:00
```

Each commit on the branch started two runs: one for the push, one for the pull request.

GitHub Actions: free on public repos, standard runners (docs.github.com/en/billing/concepts/product-billing/github-actions, read 2026-10-12).

How pass_rate works

The problem	A pass rate is only as honest as its denominator
Dropped	Cancelled, skipped, and still in progress: no verdict yet
The rate	Successes divided by the completed runs of one branch
You control	The branch
What comes out	A number from 0 to 1; a branch with no completed run raises, not 0

pass_rate: what counts as a pass

cancelled

fail-fast, or a newer push

skipped

a job that never ran

in progress

no conclusion yet

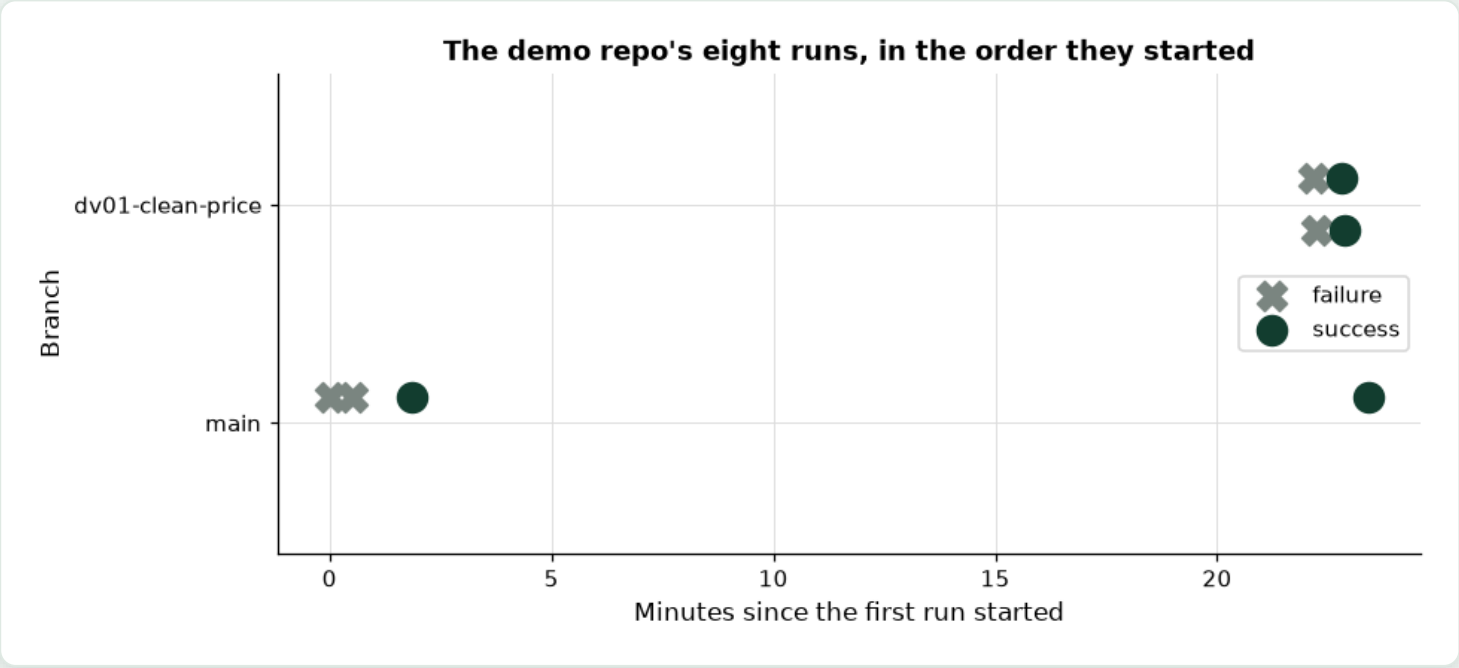
Neither a pass nor a failure: left out of both counts.

```
conclusion
failure    2
success    2
left out (cancelled, skipped, in progress): 0
pass rate on main: 0.50
a branch with no run: no completed run on docs
```

Runs, not jobs

A run's conclusion is the worst of its jobs: a failed job and a cancelled one make a failed run.

The runs, drawn



Branch	pull_request	push
dv01-clean-price	2	2
main	0	4

Each commit on a branch started two runs: the push and the pull request. A cross is a failure, a circle a success.

The case study: bondmath for a colleague

Objective

A public repo pfi-bondmath, tested on Linux on every change; one change through a pull request, tagged v1.0.

Ships

`bondmath.py, test_bondmath.py`

`the two Excel reference files`

`README.md, .gitignore, .gitattributes`

`requirements.txt: numpy, pytest`

`.github/workflows/tests.yml`

Never ships

`.env`

`my_bondmath's history`

`the work folder, remote.git`

Overview and preparation

```
file  bytes
.gitattributes     19
.github/workflows/tests.yml    453
.gitignore     40
bondmath.py  30916
bondmath_excel_reference.csv  83768
bondmath_excel_reference_tvm.csv  15082
README.md    980
requirements.txt     27
test_bondmath.py  21304
9 files, 152,589 bytes
```

Three scans, all empty

```
public_scan: problems: 0; safe to publish: yes

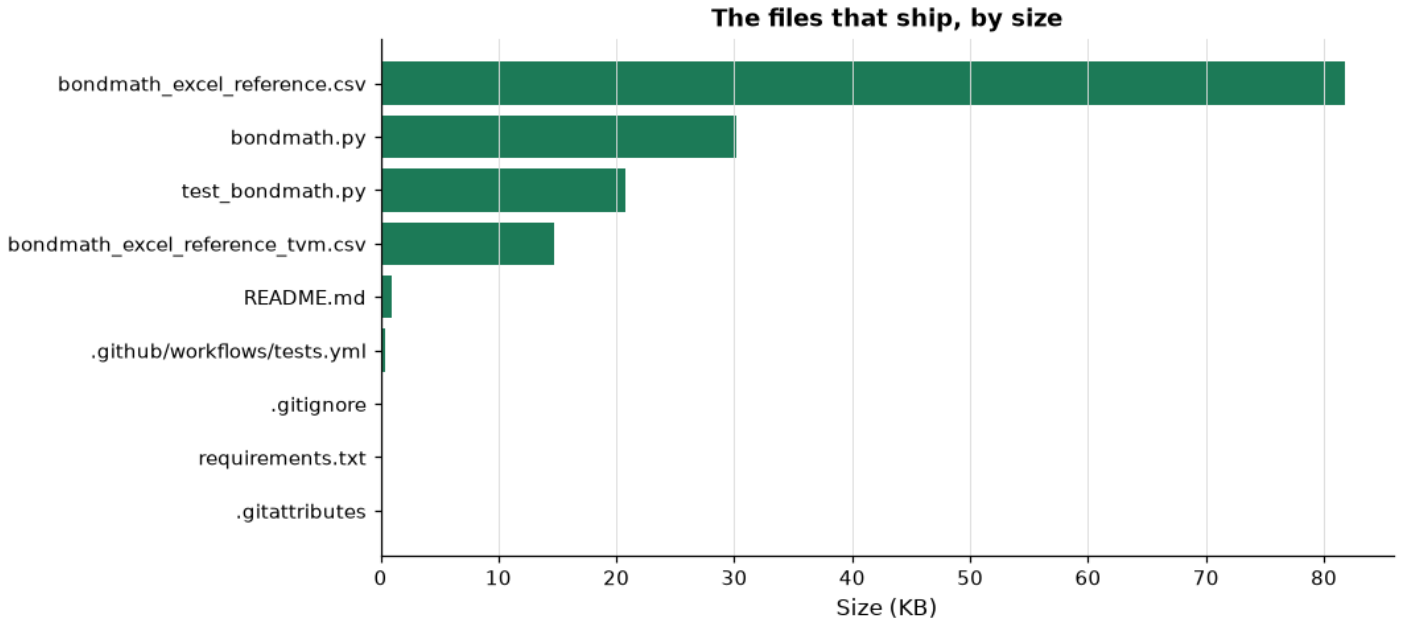
pins_check: []

check_workflow: []
```

```
exit code: 0; 40 passed in 1.89s
```

Built from named files, one by one: never a whole folder.

The files that ship



Named, one by one

code and tests

Excel's reference values

README and ignore rules

the workflow, the pins

Nothing else goes public. The sizes are this machine's, with Windows line endings.

How a pull request earns its merge

The problem	A change to main needs evidence before it merges
The checks	Every push starts the workflow, for the push and the pull request, on each Python
Red	A message from the repository about this change, not a verdict on its author
The fix	Another commit on the same branch: new runs, and the merge waits for green
What comes out	A merge that every check has seen

The change: DV01 on the clean price

Pull request: "Quote DV01 on the clean price"

The argument: the desk quotes clean prices. One line of bondmath.dv01 changes.

```
before: full_price = dirty_price(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
after:  full_price = price(settlement, maturity, coupon_pct, yield_pct, frequency, basis)
```

```
exit code: 1
FAILED test_bondmath.py::test_dv01_matches_excel_formula - AssertionError: 99...
FAILED test_bondmath.py::test_dv01_close_to_bump - AssertionError: 99080CAE8
2 failed, 38 passed in 1.68s
```

It reads well and it runs. The bump test checks how far the price actually moves.

Red on GitHub, on both Pythons

https://github.com/student/pfi-deploy-demo/pull/2

```
test (3.12)   fail   11s   (two runs x two Pythons)
test (3.12)   fail   15s   (two runs x two Pythons)
test (3.13)   fail   15s   (two runs x two Pythons)
test (3.13)   fail   15s   (two runs x two Pythons)
```

```
E  AssertionError: 99080CAE8
E  assert 0.04938208094390383 == 0.050299763430504484 ? 1.0e-08
E  AssertionError: 99080CAE8
E  assert 0.04938208094390383 == 0.050299766457797546 ? 1.0e-06
```

Which bond, which test, by how much: a red run doing its job.

The fix, the merge, v1.0

The fix: dirty price back, and the convention written down

```
* DV01 is in price points per 100 of face for one basis point, on the dirty price: modified duration x (clean price + accrued) x 0.0001,
Excel's `=MDURATION(...)*(PRICE(...)+accrued)/10000`.
```

```
test (3.12)   pass   11s
test (3.12)   pass   14s
test (3.13)   pass   14s
test (3.13)   pass   14s
```

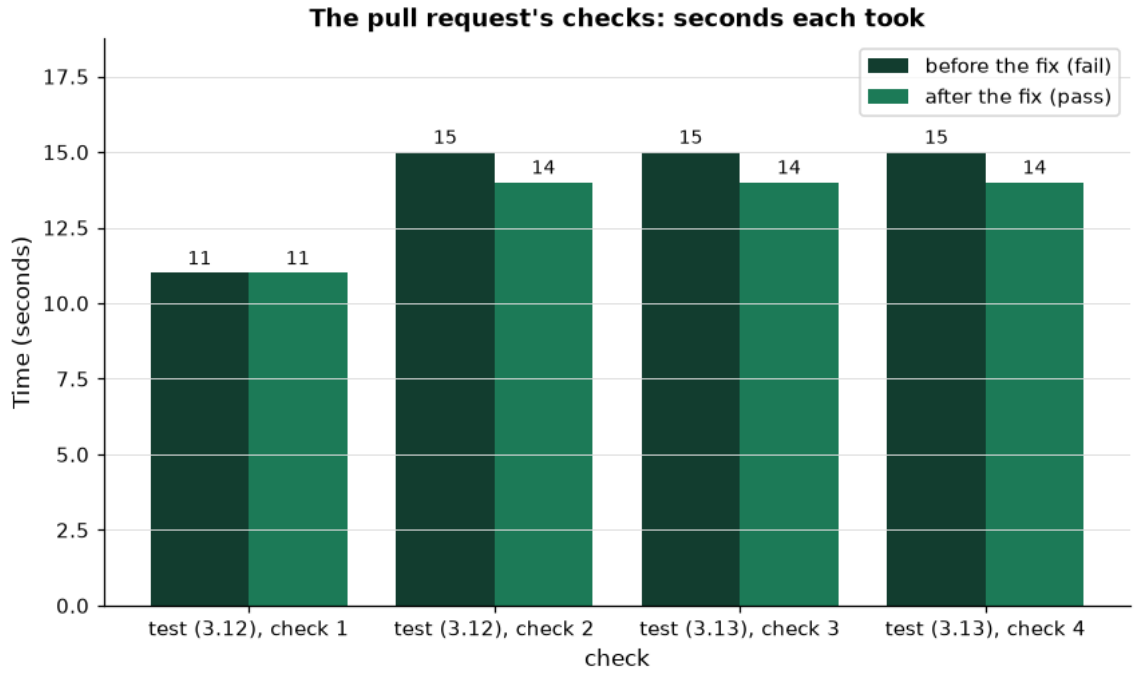
gh pr merge --merge --delete-branch

```
README.md | 1 +   (bondmath.py unchanged)
```

git tag v1.0

the commit a colleague can cite

The checks, drawn



Check	Before	After
test (3.12), check 1	11	11
test (3.12), check 2	15	14
test (3.13), check 3	15	14
test (3.13), check 4	15	14

A failing check is not faster than a passing one: the result is the whole difference.

How the comparison works

The problem	A test needs a value to compare with that does not share its mistakes
The method	Excel computed each reference row on its own; compare row by row on the same inputs
The gap	Read in the unit of the number: dollars of DV01 per 100 of face
You control	Which rows are in the reference: coupon-date bonds alone would miss a change
What comes out	A table of gaps, and a test that fails on the first gap above its tolerance

Comparison: the tests against Excel

```
case_id  excel_dv01  dirty_dv01  clean_dv01    accrued  dirty_gap  clean_gap
99080CAE8  0.05029976  0.05029976  0.04938208 1.98333333 0.00000000 0.00091768
99049XAE2  0.02191495  0.02191495  0.02111857 3.67881944 0.00000000 0.00079638
99039NAC0  0.03262745  0.03262745  0.03208614 1.66562500 0.00000000 0.00054131
99039NAG1  0.11527747  0.11527747  0.11175341 2.90937500 0.00000000 0.00352406
99034IAD4  0.06980652  0.06980652  0.06815676 2.24826389 0.00000000 0.00164975
```

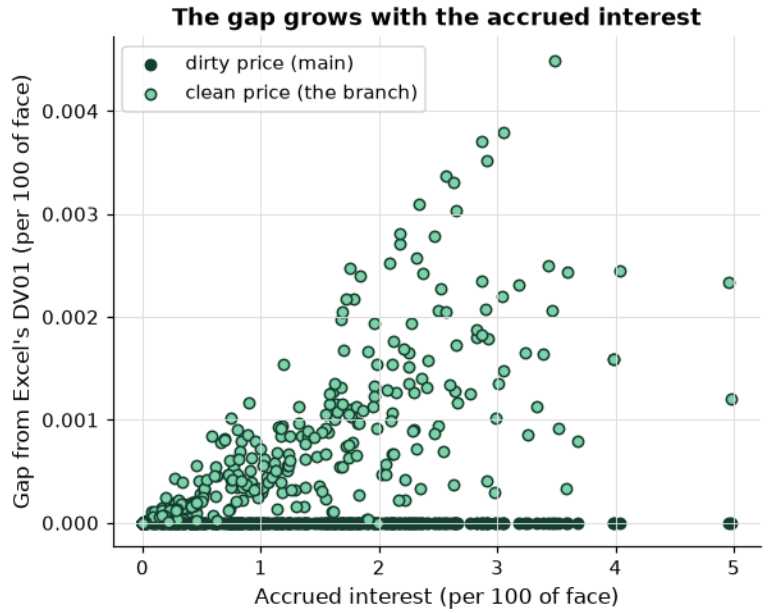
```
rows where the clean version misses the test's 1e-8: 285
rows where it passes: 21, every one with accrued 0: True
```

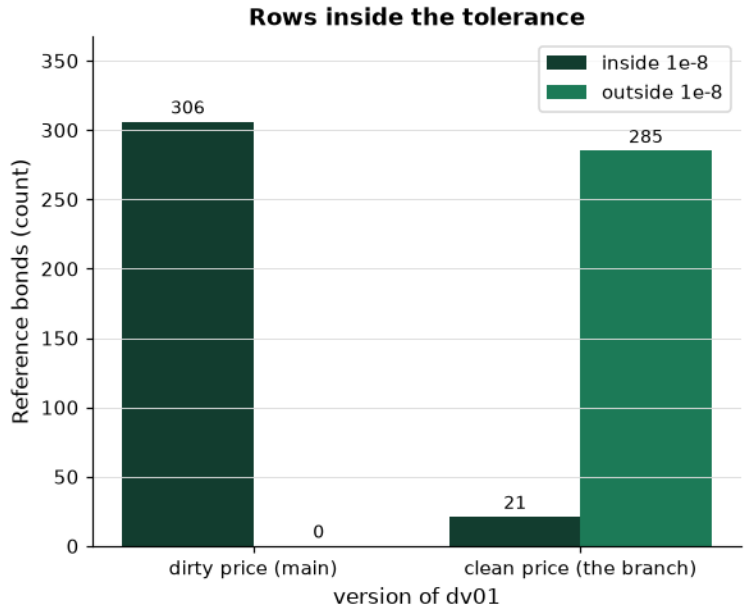
Why the dirty price

Accrued does not move with yield: DV01 = modified duration x dirty price x 0.0001.

```
99080CAE8: accrued 1.983333 per 100 of face; dirty minus clean DV01 0.00091768; modified duration x accrued x 0.0001: 0.00091768
```

The two DV01s against Excel





DV01 version	Inside	Outside
dirty price (main)	306	0
clean price (the branch)	21	285

The clean-price gap rises with the accrued interest. Only the bonds with none pass.

The workbook, and Excel side by side

```
ci_history: 8 rows, columns ['run', 'branch', 'event', 'conclusion', 'created_utc']
files_shipped: 9 rows, columns ['file', 'bytes']
read back equal to what was written: True
```

G1:	=COUNTIFS(D2:D9,"success",B2:B9,"main")	**Excel 2**
G2:	=COUNTIFS(D2:D9,"failure",B2:B9,"main")	**Excel 2**
G3:	=G1/(G1+G2)	**Excel 0.5**

Tested in Excel 16.0 build 20430 on this workbook: both counts and the rate equal pandas.

What the deployment shows, and where it fails

Where it fails

1. Green: the tests you wrote, one image, today
2. Excel's values as computed on one day
3. Only the Pythons in the matrix
4. Nothing stopped a red merge

main on the demo repo

```
gh: Branch not protected (HTTP 404)
```

A branch protection rule with required status checks makes every check pass before a merge. Off here.

A deployment without a check: SEC order, Knight Capital Americas LLC, 16 Oct 2013

"one of Knight's technicians did not copy the new code to one of the eight SMARS computer servers"

"Knight did not have a second technician review this deployment" (July 2012); losses over $460 million

Source: sec.gov/litigation/admin/2013/34-70694.pdf (Release No. 70694, File No. 3-15570), read 2026-10-12

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: a pass is anything but failure, so a cancelled run counts as a pass: 0.667 where 0.5 is right.

Code that runs is not the same as code that is right.

What you can do now

- ci_history and pass_rate on a CI history
- The files that ship, and the three scans
- A pull request: red, fixed, green, merged, v1.0
- DV01 on the dirty price, Excel as the evidence
- What green proves; protect main

Week 2, Day 1 FastAPI: A Bond Math Service